Lab-8.1

Name: D. Suraj
2303A53006
Batch:46

Lab 8: Test-Driven Development with AI – Generating and Working with Test Cases

Task 1: Password Strength Validator Task
Description

Create a function is_strong_password(password) that returns True if the password meets these rules:

- **At least 8 characters**
- **Contains at least one uppercase letter, one lowercase letter, one digit, and one special character (non-alphanumeric)**
- **No spaces**

Prompt for AI :-

"Generate at least 3 assert test cases for a function is_strong_password(password) that checks password strength according to the rules: minimum length 8, must contain uppercase, lowercase, digit, special character, and no spaces. Include edge cases."

Code (test cases + implementation)

Python:-

```python
# Test cases (AI-generated)
def test_password_validator():
    assert is_strong_password("Abcd@123") == True   # strong
    assert is_strong_password("abcd123") == False   # missing uppercase & special
    assert is_strong_password("ABCD@1234") == True  # strong
    assert is_strong_password("Abcd 123") == False  # contains space
    assert is_strong_password("Abc@12") == False    # too short

# Implementation (passes all tests)
def is_strong_password(password: str) -> bool:
    if len(password) < 8 or ' ' in password:
        return False
    if not any(c.isupper() for c in password):
        return False
    if not any(c.islower() for c in password):
        return False
    if not any(c.isdigit() for c in password):
        return False
    if not any(not c.isalnum() for c in password):
        return False
    return True

# Run tests
test_password_validator()
print("Task 1 passed")
```

Task 2: Number Classification

Task Description

Write a function classify_number(n) that returns "Positive", "Negative", or "Zero".

- **Use loops in the test (e.g., iterate over test cases)**
- **Handle invalid inputs like strings and None**

Prompt for AI (to generate test cases)

"Generate at least 3 assert test cases for a function classify_number(n). The function should return 'Positive', 'Negative', or 'Zero'. Use a loop to test multiple values, including boundary conditions (-1, 0, 1). Also include tests for invalid inputs like strings and None, expecting a TypeError."

Code (test cases + implementation)

Python:-

```python
def test_number_classifier():
    test_data = [
        (10, "Positive"),
        (-5, "Negative"),
        (0, "Zero"),
        (1, "Positive"),
        (-1, "Negative"),
    ]
    for num, expected in test_data:
        assert classify_number(num) == expected
    try:
        classify_number("hello")
        assert False, "Should have raised TypeError"
    except TypeError:
        pass
    try:
        classify_number(None)
        assert False, "Should have raised TypeError"
    except TypeError:
        pass
def classify_number(n):
    if not isinstance(n, (int, float)):
        raise TypeError("Input must be a number")
    if n > 0:
        return "Positive"
    elif n < 0:
        return "Negative"
    else:
        return "Zero"
test_number_classifier()
```

Shell

```
>>> %Run -c $EDITOR_CONTENT

  Task 2 passed
```

Task 3: Anagram Checker

Task Description

Implement is_anagram(str1, str2) that returns True if the two strings are anagrams.

- **Ignore case, spaces, and punctuation**
- **Handle edge cases like empty strings, identical words**

Prompt for AI (to generate test cases)

"Generate at least 3 assert test cases for a function is_anagram(str1, str2). The function should ignore case, spaces, and punctuation. Include edge cases such as empty strings and identical words."

Code (test cases + implementation)

Python:-

```python
# Test cases (AI-generated)
def test_anagram():
    assert is_anagram("listen", "silent") == True
    assert is_anagram("hello", "world") == False
    assert is_anagram("Dormitory", "Dirty Room") == True
    assert is_anagram("", "") == True
    assert is_anagram("a", "a") == True
    assert is_anagram("A gentleman", "Elegant man") == True
# Implementation (passes all tests)
def is_anagram(str1: str, str2: str) -> bool:
    def clean(s: str) -> str:
        return ''.join(ch.lower() for ch in s if ch.isalnum())
    return sorted(clean(str1)) == sorted(clean(str2))
# Run tests
test_anagram()
print("Task 3 passed")
```

Shell ×

```
>>> %Run -c $EDITOR_CONTENT

  Task 3 passed
```

Task 4: Inventory Class

Task Description

Create an Inventory class with methods:

- **add_item(name, quantity)**
- **remove_item(name, quantity)**
- **get_stock(name)**

Prompt for AI (to generate test cases)

“Generate at least 3 assert-based tests for an Inventory class that manages stock. The class should have methods add_item, remove_item, and get_stock. Include tests for adding, removing, and retrieving stock, as well as edge cases like removing more than available or negative quantities.” Code (test cases + implementation)

Python:-

```python
# Test cases (AI-generated)
def test_inventory():
    inv = Inventory()
    inv.add_item("Pen", 10)
    assert inv.get_stock("Pen") == 10
    inv.remove_item("Pen", 5)
    assert inv.get_stock("Pen") == 5
    inv.add_item("Book", 3)
    assert inv.get_stock("Book") == 3
    # Edge cases
    try:
        inv.remove_item("Pen", 10)  # not enough stock
        assert False
    except ValueError:
        pass
    try:
        inv.add_item("Pen", -1)     # negative quantity
        assert False
    except ValueError:
        pass
# Implementation (passes all tests)
class Inventory:
    def __init__(self):
        self._stock = {}
    def add_item(self, name: str, quantity: int) -> None:
        if quantity < 0:
            raise ValueError("Quantity cannot be negative")
        self._stock[name] = self._stock.get(name, 0) + quantity
    def remove_item(self, name: str, quantity: int) -> None:
        if quantity < 0:
            raise ValueError("Quantity cannot be negative")
        if name not in self._stock or self._stock[name] < quantity:
            raise ValueError(f"Not enough stock for '{name}'")
        self._stock[name] -= quantity
        if self._stock[name] == 0:
            del self._stock[name]
    def get_stock(self, name: str) -> int:
        return self._stock.get(name, 0)
# Run tests
test_inventory()
print("Task 4 passed")
```

Shell

```
>>> %Run -c $EDITOR_CONTENT

Task 4 passed
```

Task 5: Date Validation & Formatting
Task Description:-
Write validate_and_format_date(date_str) that:

- **Validates the string in MM/DD/YYYY format**
- **Returns "Invalid Date" if invalid**
- **Otherwise returns the date as YYYY-MM-DD**

Prompt for AI :-
"Generate at least 3 assert test cases for a function validate_and_format_date(date_str). It should validate dates in MM/DD/YYYY format and convert valid dates to YYYY-MM-DD. Include tests for invalid dates (like Feb 30) and edge cases (leap years, wrong month/day)."

Code (test cases + implementation)
Python:-

```python
# Test cases (AI-generated)
def test_date_validator():
    assert validate_and_format_date("10/15/2023") == "2023-10-15"
    assert validate_and_format_date("02/30/2023") == "Invalid Date"
    assert validate_and_format_date("01/01/2024") == "2024-01-01"
    assert validate_and_format_date("13/01/2024") == "Invalid Date"
    assert validate_and_format_date("02/29/2024") == "2024-02-29"  # leap year
    assert validate_and_format_date("02/29/2023") == "Invalid Date" # non-leap
# Implementation (passes all tests)
import re
from datetime import datetime
def validate_and_format_date(date_str: str) -> str:
    if not re.match(r'^\d{2}/\d{2}/\d{4}$', date_str):
        return "Invalid Date"
    try:
        month, day, year = map(int, date_str.split('/'))
        datetime(year, month, day)  # raises ValueError if invalid date
        return f"{year:04d}-{month:02d}-{day:02d}"
    except ValueError:
        return "Invalid Date"
# Run tests
test_date_validator()
print("Task 5 passed")
```

```
hell ×
>> %Run -c $EDITOR_CONTENT

Task 4 passed
```